

Docker Network

Container-to-container communication • Names • IP addresses

1. Objective

This practical demonstrates communication between two Docker containers connected to the same user-defined Docker network. It verifies communication using both container names and container IP addresses.

2. Docker Network Concepts

A Docker network provides connectivity between containers. On a user-defined bridge network, containers can communicate with each other using container names through Docker's built-in DNS service. Each container also receives an IP address on the network, which can be used for direct communication.

3. Create a User-Defined Network

```
docker network create my-network  
  
docker network ls  
docker network inspect my-network
```

The command creates a bridge network named **my-network**. Containers attached to this network can communicate with one another.

4. Start Two Containers

```
docker run -dit --name container1 --network my-network alpine sh  
docker run -dit --name container2 --network my-network alpine sh  
  
docker ps
```

Both containers are attached to the same user-defined network. The `-d` option runs them in the background, `-i` keeps standard input open, and `-t` allocates a terminal.

5. Communication Using Container Names

First, install a small network utility in container1 and test the hostname container2:

```
docker exec container1 apk add --no-cache curl  
  
docker exec container1 curl http://container2
```

The request resolves **container2** to the IP address assigned to that container and attempts to connect. For a simple HTTP response demonstration, you can instead run an HTTP server in container2:

```
docker exec container2 apk add --no-cache python3  
  
docker exec -d container2 python3 -m http.server 8000  
  
docker exec container1 curl http://container2:8000
```

The successful response demonstrates that container1 can reach container2 by its Docker container name.

6. Find the Container IP Address

```
docker inspect -f '{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}' container2
```

Suppose the command returns an address such as **172.18.0.3**. The exact address depends on Docker's network configuration and must be obtained from your environment rather than assumed.

7. Communication Using the Container IP

```
# Replace CONTAINER2_IP with the IP returned above
docker exec container1 curl http://CONTAINER2_IP:8000
```

If the network and service are working correctly, container1 receives the same HTTP response using container2's IP address instead of its name.

8. Verify Network Membership

```
docker network inspect my-network
```

The network inspection output lists connected containers and their network details, including IP addresses. This is useful for verifying that both containers are attached to the intended network.

9. Name vs. IP Address

Method	Example	Advantage
Container name	http://container2:8000	Stable and easy to use; Docker DNS resolves the name.
Container IP	http://172.18.0.3:8000	Direct addressing of the current network IP.

Container names are generally preferred for application-to-application communication because container IP addresses can change when containers are recreated. IP addressing is useful for troubleshooting and for demonstrating the underlying network connectivity.

10. Cleanup

```
docker stop container1 container2
docker rm container1 container2
docker network rm my-network
```

11. Expected Results

Both containers should appear as members of my-network. Container1 should be able to contact container2 using the hostname container2. After retrieving container2's actual Docker IP address, container1 should also be able to contact the HTTP service using that IP address.

Conclusion

Docker user-defined networks provide isolated and convenient communication between containers. Docker's internal DNS allows containers to find one another by name, while each container also receives an IP address for direct network communication. In practical multi-container applications, service names are usually preferable to hard-coded IP addresses.